PHASE 2 — Embeddings and In-Memory Semantic Retrieval

Objective

Implement embedding generation and semantic retrieval over the chunks produced in Phase 1.

The vector index must be held in memory using NumPy.

Step 1 — Understand the embedding model

Inspect the existing LLM configuration.

Use:

- OPENAI_API_KEY
- OPENAI_BASE_URL
- OPENAI_EMBEDDING_MODEL

Do not assume the embedding model name.

Verify that the configured endpoint supports embedding generation.

Use the OpenAI Python SDK with the provided OpenAI-compatible endpoint.

Never hardcode credentials.

Step 2 — Implement embedding generation

Create an embedding component responsible for:

- Accepting document chunks.
- Sending their text to the embedding endpoint.
- Handling API errors.
- Validating embedding responses.
- Preserving the association between each embedding and its original chunk.

Generate embeddings in batches where appropriate.

Avoid unnecessary API calls.

For this assignment, embeddings may be generated when the application starts. Persistent caching is optional and must be justified before implementation.

Step 3 — Implement in-memory vector storage

Use NumPy.

The vector store should maintain:

- An embedding matrix.
- The corresponding document chunks.
- Their metadata.
- Any necessary mapping between vector positions and chunk identifiers.

Ensure all embeddings have consistent dimensions.

Step 4 — Implement similarity search

Implement cosine similarity.

The retrieval system must:

1. Accept a user query.
2. Generate its embedding.
3. Compare it with stored document embeddings.
4. Rank the results.
5. Return the top-K relevant chunks.

Handle:

- Empty queries.
- Empty indexes.
- Invalid embeddings.
- Dimension mismatches.
- Similarity ties.
- Configurable top-K values.

Do not confuse similarity scores with confidence or factual correctness.

Step 5 — Tests

Write tests for:

- Cosine similarity.
- Vector normalization.
- Ranking.
- Top-K behavior.
- Empty indexes.
- Dimension mismatches.
- Metadata preservation.
- Embedding API failures.

Mock external embedding API calls in unit tests.

Do not make real API calls during ordinary unit tests.

Constraints

- Do not implement the agent.
- Do not implement tool calling.
- Do not implement the critic.
- Do not introduce an external vector database.
- Do not introduce unnecessary frameworks.

Deliverables

Explain:

1. How embeddings are generated.
2. How the vector matrix is structured.
3. How cosine similarity works in this implementation.
4. How query retrieval works.
5. Limitations of the approach.
6. Test results.